

Documento Técnico: Auditoría y Rediseño del Sistema Bancario Legacy

1. Hallazgos de la auditoría de memoria

Al analizar la función `crearCuenta()` del código original, se identificaron dos fugas de memoria principales:

Fuga 1 - Puntero titular: dentro de `crearCuenta()` se reserva memoria dinámica con `new char[50]` para almacenar el nombre del titular. Sin embargo, en ningún punto del programa se libera esta memoria con `delete[]`. Cada vez que se crea una cuenta, esos 50 bytes quedan reservados de forma permanente mientras el programa esté en ejecución.

Fuga 2 - Struct `CuentaLegacy`: el struct completo se reserva con `new CuentaLegacy()` dentro de la misma función, pero tampoco se libera con `delete` en ningún momento, esto puede llegar a generar una fuga en cascada.

3. Problemas de arquitectura identificados

Al analizar `procesarRetiro()` se identificó que el campo `tipoCuenta` (1 o 2) se usa para decidir, mediante una cadena de `if/else if`, qué regla de negocio aplicar según el tipo de cuenta, esto viola el Principio Abierto/Cerrado (OCP) de SOLID: si en el futuro se necesita agregar un nuevo tipo de cuenta sería obligatorio modificar la función `procesarRetiro()` existente.

También se observó una violación al Principio de Responsabilidad Única (SRP): la función `procesarRetiro()` concentra la lógica de validación de todos los tipos de cuenta en un solo lugar, en vez de que cada tipo de cuenta sea responsable de su propia regla de retiro.

Por último, se identificó que el sistema no cuenta con ningún registro de auditoría, confirmando que no existe trazabilidad de las operaciones realizadas.

4. Justificación del rediseño

- `CuentaBancaria` (clase abstracta): encapsula los atributos comunes (titular, saldo, `numeroCuenta`) como privados, exponiéndolos únicamente a través de getters y setters. Define el método abstracto `aplicarComisionMensual()`, obligando a cada subclase a implementar su propia regla de negocio.
- `CuentaAhorros` y `CuentaCorriente`: cada una sobrescribe `aplicarComisionMensual()` con su propia lógica eliminando los condicionales.

Resuelve:

- Gestión de memoria: al usar `String` en vez de `char*` y dejar que la JVM administre el ciclo de vida de los objetos mediante el Garbage Collector, se elimina por completo el riesgo de fugas de memoria manual presentes en el código original.
- Cumplimiento de OCP: agregar un nuevo tipo de cuenta ahora solo requiere crear una nueva subclase de `CuentaBancaria`, sin modificar las clases existentes.
- Cumplimiento de SRP: cada clase es responsable únicamente de su propia lógica de negocio.

- Encapsulamiento: los atributos ya no son accesibles directamente desde fuera de la clase, previniendo modificaciones no controladas del estado de la cuenta.

Se implementó la clase `RegistroAuditoriaBancaria`, que implementa la interfaz `AutoCloseable` y se utiliza mediante `try-with-resources`, lo que garantiza que el recurso de auditoría se cierre correctamente al finalizar su uso, sin depender del Garbage Collector para liberar recursos.

5. Conclusión

La migración del sistema bancario de C++ a Java, junto con el rediseño orientado a objetos, corrige tanto las fugas de memoria como las violaciones a los principios SOLID detectadas en el código original. El nuevo diseño es más seguro y extensible permitiendo incorporar nuevos tipos de cuenta o reglas de negocio sin arriesgarla estabilidad del código existente.